

Digital Design (F215) — Combinational Circuits Master Notes

Every building block, its types, truth tables, equations and circuit diagrams

1. Number Systems & Conversion

- Value = $\sum d_i r^i$; binary weights (right $\rightarrow$ left) 1, 2, 4, 8, 16, 32, $\dots$, 2^k . n bits $\Rightarrow 2^n$ patterns (0 to $2^n - 1$). Store N items $\Rightarrow \lceil \log_2 N \rceil$ bits.
- Hex digit = 4 bits, octal digit = 3 bits; **group from the binary point**, pad far group with 0s ($A=10 \dots F=15$).
- Dec $\rightarrow$ bin**: integer — divide by 2, remainders are bits **LSB first**; fraction — multiply by 2, integer-outs are bits after the point.

2. Signed Numbers & Overflow

- 2's complement** (used everywhere): negate = invert all +1. MSB weight is -2^{n-1} ; range -2^{n-1} to $+2^{n-1} - 1$. 4-bit: $-8 \dots +7$. Sign-extend by copying the MSB.
- 1's comp** negate = invert; **sign-magnitude** MSB=sign (both have ± 0).
- Unsigned add** wrong $\Leftrightarrow C_n = 1$. **Unsigned sub** ($A+B'+1$): $C_n = 1 \Rightarrow A \geq B$; $C_n = 0 \Rightarrow A < B$ (result is 2's comp of $B-A$).
- Signed overflow** $V = C_{n-1} \oplus C_n$. $A +$ and a $-$ number never overflow.
- Bit-width detective**: wrong output = true value mod 2^n ; a correct row lower-bounds n , a wrong row fixes 2^n .

3. Binary Codes

- BCD (8421)**: one decimal digit per 4 bits; 1010–1111 never occur $\Rightarrow$ don't-cares $d(10-15)$.
- Excess-3**: code of n is $(n+3)_2$; self-complementing (invert bits $\Rightarrow$ code of $9-n$).
- Gray**: one bit changes between neighbours (incl. wrap); not weighted. $g_i = b_{i+1} \oplus b_i$; $b_i = b_{i+1} \oplus g_i$ (circuits in §20).
- Parity**: $P_{\text{even}} = d_3 \oplus d_2 \oplus d_1 \oplus d_0$; checker = XOR of all bits. Catches any odd number of flips.
- Gray input in a K-map**: list rows in Gray order, place each by its binary value.

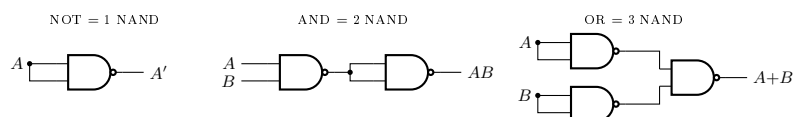
4. Boolean Algebra

- Identity $x+0=x$, $x \cdot 1=x$. Null $x+1=1$, $x \cdot 0=0$. Idempotent $x+x=x$. Complement $x+x'=1$; $(x')'=x$.
- Absorption $x+xy=x$; redundant literal $x+x'y=x+y$.
- Distributive $x(y+z)=xy+xz$ **and** $x+yz=(x+y)(x+z)$.
- De Morgan $(x+y)'=x'y'$, $(xy)'=x'+y'$. Consensus

9. Logic Gates — the atoms

																																																			
AND AB	OR $A+B$	NOT A'	NAND	NOR	XOR $A\oplus B$	XNOR																																													
<table><tr><th>A</th><th>B</th><th>AN</th><th>OR</th><th>NA</th><th>NO</th><th>XO</th><th>XN</th><th>A'</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td></tr></table>							A	B	AN	OR	NA	NO	XO	XN	A'	0	0	0	0	1	1	0	1	1	0	1	0	1	1	0	1	0	1	1	0	0	1	1	0	1	0	0	1	1	1	1	0	0	0	1	0
A	B	AN	OR	NA	NO	XO	XN	A'																																											
0	0	0	0	1	1	0	1	1																																											
0	1	0	1	1	0	1	0	1																																											
1	0	0	1	1	0	1	0	0																																											
1	1	1	1	0	0	0	1	0																																											

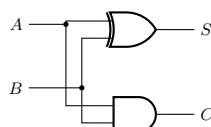
- AND = 1 iff all 1; OR = 1 iff any 1; XOR = 1 iff odd #1s (parity / "inputs differ"); XNOR = 1 iff equal.
- Controlling values**: $A \cdot 1 = A$, $A \cdot 0 = 0$ (AND enables/blocks); $A + 1 = 1$; $A \oplus 0 = A$, $A \oplus 1 = A'$ (XOR = controlled inverter).
- NAND / NOR are universal** — every block below can be built from one gate type:



10. Adders

▷ **Half adder** — adds two bits, no carry-in

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



$$S = A \oplus B$$
$$C = AB$$

Carry = AND, sum = XOR.

▷ **Full adder** — adds three bits (A, B, C_{in})

$$xy + x'z + yz = xy + x'z.$$

- Precedence NOT > AND > OR. $(AB)' \neq A'B'$.

5. Minterms, Maxterms, Canonical Forms

- Row i : first-named variable is the MSB. **Minterm** m_i (AND, var = 0 $\rightarrow$ primed) is 1 only at row i ; **maxterm** M_i (OR, var = 1 $\rightarrow$ primed) is 0 only at row i ; $M_i = m'_i$.
- SOP** = $\Sigma m(1\text{-rows})$; **POS** = $\Pi M(0\text{-rows})$; the two lists are complementary; $F' = \Sigma m(0\text{-rows})$.
- SOP = 2-level AND-OR; POS = 2-level OR-AND (≤ 2 gate delays).

6. Karnaugh Maps

4-var numbering (rows AB , cols CD , Gray order):

$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

- Group 2^k cells (rectangle, may wrap edges & corners) $\Rightarrow$ drops k variables; vars constant in the group survive (0 $\rightarrow$ primed). Make groups as large/few as possible.
- PI** cannot be enlarged; **EPI** covers a 1 no other PI covers (must use). Procedure: all PIs $\rightarrow$ EPis $\rightarrow$ cover rest with fewest PIs.
- Don't-cares**: treat X as 1 only if it grows a group. **POS**: group 0s $\rightarrow F' \rightarrow$ De Morgan. 5-var = two 4-var maps ($A=0, 1$).

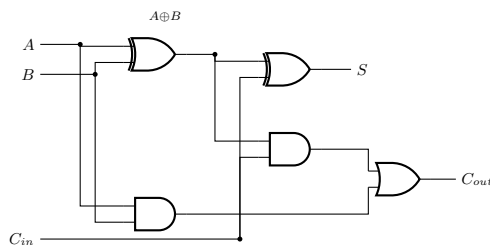
7. Quine–McCluskey

- List minterms + **don't-cares** grouped by #1s; merge pairs differing in one bit ($-$ at that position); unticked terms = PIs.
- PI chart**: columns = required minterms only; single- $\times$ column $\Rightarrow$ essential PI.

8. NAND / NOR & Bubble Pushing

- Universal. NAND: NOT = 1, AND = 2, OR = 3; NOR: NOT = 1, OR = 2, AND = 3.
- SOP $\rightarrow$ NAND-NAND**, **POS $\rightarrow$ NOR-NOR** (double-bubble middle wires). Single-literal term needs its complement at level 2.
- Need NAND $\Rightarrow$ minimise SOP; need NOR $\Rightarrow$ minimise POS. For 2-input gates: factor and share sub-terms first.

A	B	C_{in}	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

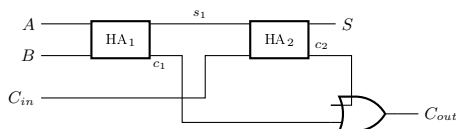


$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

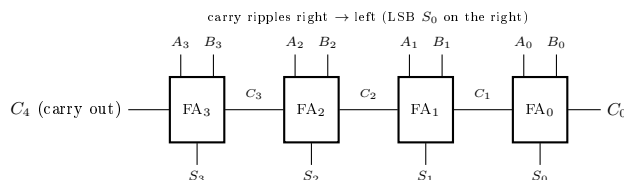
= majority.
S-map is a checkerboard $\Rightarrow$ XOR.

▷ Full adder from two half adders + OR



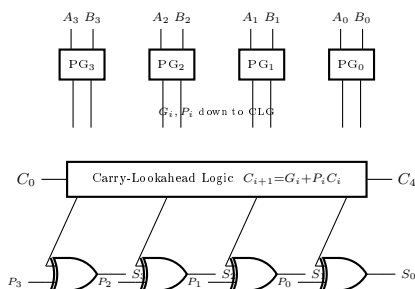
HA1(A, B) $\rightarrow (s_1, c_1)$; HA2(s_1, C_{in}) $\rightarrow (S, c_2)$; $C_{out} = c_1 + c_2$.
The two carries are never 1 together, so OR (or XOR) works. *Where used:* the cell of every ripple / CLA / BCD / array-multiplier adder.

▷ n -bit ripple-carry adder — chain n full adders



Ripple delay: worst case = $2n+1$ gate levels ($O(n)$); 64-bit = 129 delays. The carry must settle before the next clock edge, so this sets the max adder speed. *Max frequency* $\approx 1/(2n+1)t_{pd}$.

▷ Carry-lookahead adder (CLA) — removes the ripple



Generate $G_i = A_i B_i$, propagate $P_i = A_i \oplus B_i$. Then every carry is a flat 2-level function:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

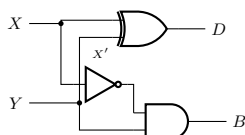
Delay = constant **4 gate levels** (any n); gate count $O(n^2)$. $S_i = P_i \oplus C_i$. Borrow-lookahead: $G_i = X'_i Y_i$, $P_i = X'_i + Y_i$.

11. Subtractors

▷ Half subtractor — computes $X - Y$

X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

(B=borrow, D=diff)

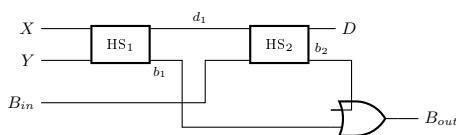


$$D = X \oplus Y$$

$$B = X'Y$$

Borrow B is *not* symmetric: $X'Y \neq Y'X$ (needs an inverter on X , unlike the adder's carry AB).

▷ Full subtractor — $X - Y - B_{in}$ (from two half subtractors)



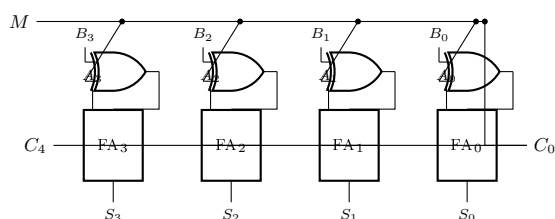
$$D = X \oplus Y \oplus B_{in}; \quad B_{out} = X'Y + X'B_{in} + YB_{in} = X'Y + (X' + Y)B_{in}.$$

HS1(X, Y) $\rightarrow (d_1, b_1)$; HS2(d_1, B_{in}) $\rightarrow (D, b_2)$; $B_{out} = b_1 + b_2$.
Wiring trap: HS2 is ($d_1 - B_{in}$), i.e. $.x(d_1), .y(B_{in})$ — swapping them makes B_{out} wrong at (0, 0, 1).

▷ n -bit subtractor = adder with 2's complement

To compute $A - B$: invert every bit of B and set $C_0 = 1$ (adds the +1), then use a normal ripple/CLA adder: $A + B' + 1$. This is exactly the adder/subtractor below with $M = 1$. Borrow-lookahead mirrors carry-lookahead with $G_i = X'_i Y_i$, $P_i = X'_i + Y_i$ (OR-type propagate).

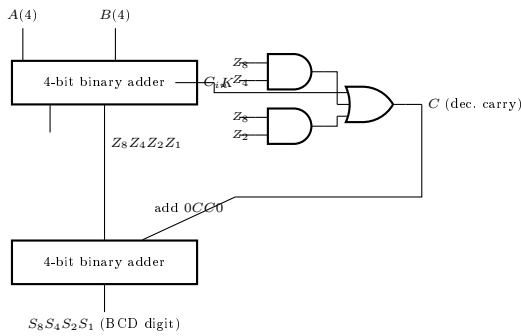
12. Adder / Subtractor (one circuit does both)



Each B_i passes through an XOR with control M (XOR = controlled inverter); M also feeds C_0 .

- $M=0$: B_i unchanged, $C_0=0 \Rightarrow A + B$.
 - $M=1$: $B_i \rightarrow B'_i$, $C_0=1 \Rightarrow A + B' + 1 = A - B$.
- Overflow $V = C_{n-1} \oplus C_n$ (same circuit for both modes). This is the standard way to build a subtractor.

13. BCD (Decimal) Adder



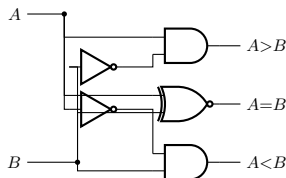
Add two BCD digits $+C_{in}$ with a binary adder $\rightarrow K Z_8 Z_4 Z_2 Z_1$.

- If the sum ≤ 9 : answer is correct, $C=0$.
- If > 9 (i.e. $K=1$ or the 4-bit value is 1010–1111): **add 0110 (6)** to skip the six unused codes, and set the decimal carry.

Correction $C = K + Z_8 Z_4 + Z_8 Z_2$. The 2nd adder adds $0CC0$; its own carry is ignored. n -digit = n stages, carry $\rightarrow$ carry.

14. Magnitude Comparators

▷ 1-bit comparator



$$A > B = AB', \quad A < B = A'B, \quad A = B = (A \oplus B)' \text{ (XNOR)}.$$

n -bit comparator (dictionary order from the MSB): equal bit $x_i = (A_i \oplus B_i)'$.

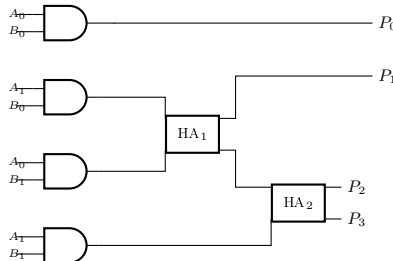
$$A = B = x_3 x_2 x_1 x_0$$

$$A > B = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$A < B$ is symmetric (swap $A \leftrightarrow B$). Term $x_3 x_2 A_1 B_1'$ = “bits 3,2 equal, bit 1 has $A=1, B=0$ ”.

15. Multipliers

▷ 2×2 array multiplier ($A_1 A_0 \times B_1 B_0$)

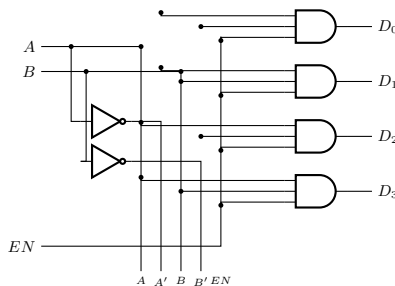


Partial product = multiplicand AND one multiplier bit.

- $P_0 = A_0 B_0$ (one term, no add).
 - $P_1 = \text{HA of } A_1 B_0, A_0 B_1$.
 - $P_2 = \text{HA of } A_1 B_1 \text{ and } P_1 \text{'s carry; } P_3 = \text{that carry.}$
- $J \times K$: $J \cdot K$ AND gates, $(J-1)$ K -bit adders, product has $J+K$ bits. Multiply by const 6 = $\times 4 + \times 2$ (shift & add).

16. Decoders

A decoder turns an n -bit code into 2^n lines, exactly one active; **each output is a minterm**. ▷ **2-to-4 decoder (active-high) with enable — gate level**

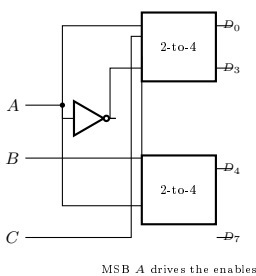


$$D_0 = A'B', D_1 = A'B, D_2 = AB', D_3 = AB \text{ (all ANDed with EN)}.$$

EN	A	B	D ₀	D ₁	D ₂	D ₃
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

- 2^n AND gates + inverters. $EN=0 \Rightarrow$ all outputs 0.
- **Active-high** outputs = minterms m_k . **Active-low** (NAND-based, bubbled outputs) = maxterms M_k .
- **Decoder + enable = demultiplexer** (§19).

▷ Bigger decoders + implementing functions



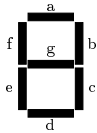
3-to-8 from two 2-to-4: B, C go to both; A enables the upper (A' , giving D_0 – D_3) or lower (A , giving D_4 – D_7). (4-to-16 = two 3-to-8, etc.)

Implement any F with one decoder — pick the cheapest:

- active-high + **OR** of the 1-rows; or + **NOR** of the 0-rows;
- active-low + **NAND** of the 1-rows; or + **AND** of the 0-rows.

Gate inputs = min(#1s, #0s). One decoder serves several functions (one OR each). Full adder: $S = \Sigma m(1, 2, 4, 7)$, $C = \Sigma m(3, 5, 6, 7)$ from one 3-to-8 + two ORs.

▷ BCD-to-7-segment decoder (display driver)

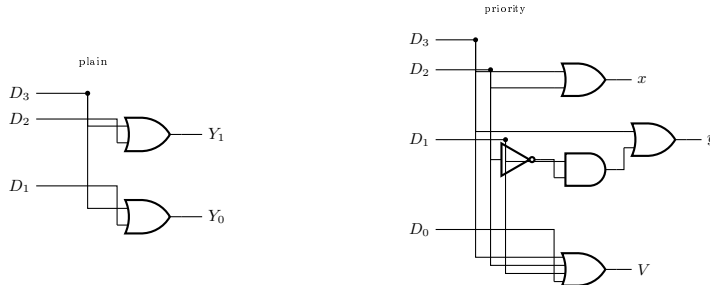


digit	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

Input = one BCD digit (4 bits), outputs = 7 segment lines *a-g*. Inputs 10-15 are **don't-cares** $\Rightarrow$ design **one K-map per segment** (7 maps). *Common-cathode* display = active-high outputs; *common-anode* = active-low (segment lit when output = 0). E.g. segment $a = \Sigma m(0, 2, 3, 5, 6, 7, 8, 9) + d(10-15)$.

17. Encoders

The reverse of a decoder: 2^n inputs $\rightarrow n$ -bit index of the active line. $\triangleright$ **4-to-2 encoder (plain)** and **4-to-2 priority encoder**



Plain: $Y_1 = D_2 + D_3$, $Y_0 = D_1 + D_3$. *Fails* if two inputs active (OR of codes) or none active ($= D_0$). **Priority** (highest index wins + Valid V ; x in input col = don't-care):

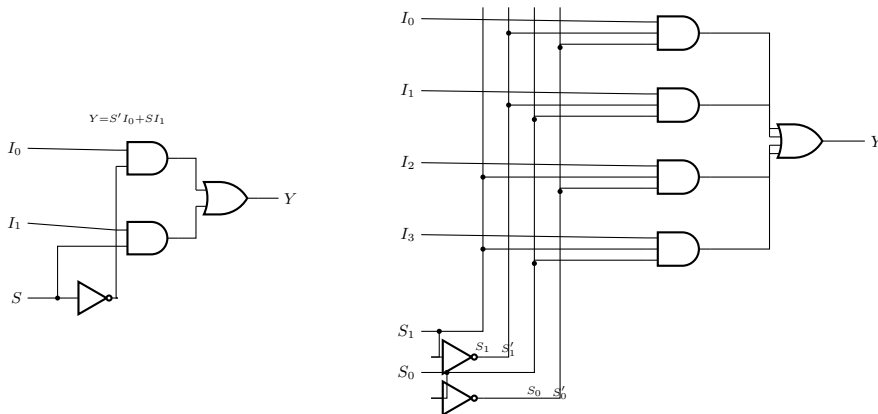
D_3	D_2	D_1	D_0	x	y	V
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

$x = D_2 + D_3$, $y = D_3 + D_1 D'_2$, $V = D_0 + D_1 + D_2 + D_3$. 8-to-3: $Y_2 = D_4 + D_5 + D_6 + D_7$, etc.

Decoder $\rightarrow$ wires $\rightarrow$ encoder = any 1-to-1 code converter.

18. Multiplexers (data selectors)

Select lines choose which data input reaches the output: $Y = \sum_k m_k(S) I_k$. $\triangleright$ **2-to-1** and **4-to-1 MUX** — gate level

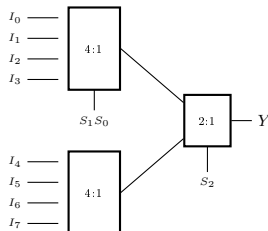


$$4:1: Y = S'_1 S'_0 I_0 + S'_1 S_0 I_1 + S_1 S'_0 I_2 + S_1 S_0 I_3.$$

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Internally a 2-to-4 decoder whose lines gate the data. Data 0,1,1,0 $\Rightarrow$ XOR. Active-low enable: disabled $\Rightarrow Y=0$.

$\triangleright$ **Bigger MUX, and implementing functions with a MUX**



8:1 from two 4:1 + one 2:1: low bits $S_1 S_0$ feed both 4:1s; MSB S_2 picks between them. **Three ways to realise F :**

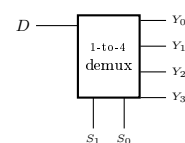
- n vars on selects: tie each I_k to that row's output (0/1) — a lookup.
 - $n-1$ vars on selects (var D left over): $I_k \in \{0, 1, D, D'\}$ via the implementation table (write minterms in a D' row and a D row, circle 1s, read columns).
 - $n-2$ vars on selects: each I_k is a function of the 2 leftover vars (K-map that column).
- " AB as $s_1 s_0$ " $\Rightarrow A$ is the MSB select. Only-true-inputs trick: feed leftover vars to a decoder for complemented terms.

19. Demultiplexers

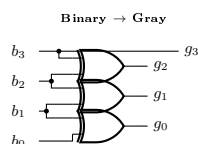
A **demux** routes one data line D to one of 2^n outputs chosen by the selects: $Y_k = D \cdot m_k(S)$. It is **exactly a decoder with the data wired to the enable** ($Y_0 = D S'_1 S'_0$, $Y_1 = D S'_1 S_0$, $Y_2 = D S_1 S'_0$, $Y_3 = D S_1 S_0$).

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

So one 3-input-AND decoder drawing serves as a 1-to-4 demux with $EN \rightarrow D$. Use: send a serial stream to selected registers; the receive side of a mux link; building larger decoders.



20. Code Converters (XOR circuits)

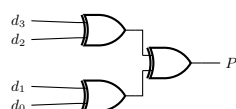


Bin→Gray: $g_3=b_3$, $g_i=b_{i+1} \oplus b_i$ (parallel XORs).

Gray→Bin: $b_3=g_3$, $b_i=b_{i+1} \oplus g_i$ (*cascaded* — each needs the bit above).

BCD→Excess-3: 4 in / 4 out, $d(10-15)$; one K-map per output: $z=D'$, $y=(C \oplus D)'$, etc. (general code-converter procedure: truth table → one K-map per output).

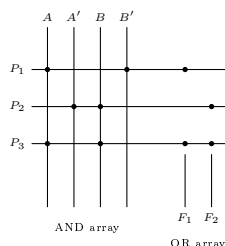
▷ Parity generator / checker (XOR tree)



Even-parity generator: $P = d_3 \oplus d_2 \oplus d_1 \oplus d_0$ (a tree of XORs; output = 1 when the data has an odd number of 1s, making the total even).

Checker at the receiver: XOR all received bits incl. P ; result = 1 flags an error. Detects any **odd** number of flipped bits (one flip caught; two cancel). Odd-parity uses an XNOR at the final stage.

21. Programmable Logic (ROM / PLA / PAL)



Every SOP = AND array (products) feeding OR array (sums); $\times/\cdot$ = a programmed connection.

- **ROM:** AND array fixed (full decoder, all 2^n minterms), OR programmable = stored truth table ($2^n \times m$).
- **PLA:** both arrays programmable; product terms **shared** across outputs. Size = in $\times$ #terms $\times$ out; “min size” = fewest distinct terms (share, even non-prime terms).
- **PAL:** AND programmable, OR fixed (own ANDs per output) $\Rightarrow$ minimise each output separately.
- **Programming table:** per term, input col 1/0/– = true/complemented/unused; output col 1 = term feeds that OR.

22. Which Block for Which Job (quick index)

You need to...	Use
add / subtract numbers	full adders (ripple or CLA); adder/subtractor with mode M
add decimal digits	BCD adder (binary adder + +6 correction)
turn a code into one active line	decoder (output = minterm)
build any SOP from a code	decoder + OR, or a MUX, or a PLA
select 1 of many data lines	multiplexer (n selects, 2^n inputs)
route 1 line to many places	demultiplexer = decoder + enable
compress active line $\rightarrow$ number	encoder (priority if > 1 may be active)
compare two numbers	magnitude comparator (XNOR per bit)
multiply	AND array + adders ($J \cdot K$ ANDs)
make only true/complement inputs	inverters, or a decoder (free minterms)
cheap custom two-level logic	PLA (shared terms) / ROM (lookup) / PAL

Open-book checklist: (1) truth table first, mark don't-cares; (2) label K-map axes in Gray order, confirm the MSB; (3) for “use only these blocks”, first write what each block computes, then map the pieces of F ; (4) watch enable/output **bubbles** (active-low) and **select order** ($s_1 s_0$); (5) marking is often all-or-none — recheck one cell/one wire before moving on.